ALHAZ MOCKBUL HOSSAIN COLLEGE



Software Management Project Book

Course Name: Software Engineering LAB
Course Code: 530220

Project: **StayEase – Online Reservation and Booking System**

Submitted To: Sharmin Akter Jui

Lecturer, CSE Department
Alhaz Mockbul Hossain College

Submitted By: _____

Roll: _____

Reg: _____

Department: Department of Computer Science & Engineering (CSE)

Date: _____

_____

Signature

Table of Contents

Chapter 1: Introduction **2**

1.1 Project Overview 2

1.2 Background and Problem Statement 2

1.3 Objectives 3

1.4 Scope of the Project 3

1.5 Target Users 3

1.6 Key Features 4

1.7 Organization of the Book 4

Chapter 2: Methodology **5**

2.1 Development Approach 5

2.2 Project Phases 6

2.3 Requirement Analysis 6

2.4 Tools and Technologies 8

2.5 System Architecture 8

2.6 Key Technical Decisions 9

2.7 Testing Approach 9

2.8 Deployment 10

Chapter 3: Design **11**

3.1 Flow Chart 11

3.2 E-R Diagram 13

3.3 Data Flow Diagram (DFD) 15

Chapter 4: Implementation **18**

4.1 Project Structure 18

4.2 Database Model 19

4.3 Availability Check 19

4.4 Booking Transaction (No Double Booking) 20

4.5 Input Validation 22

4.6 Server Action for Booking 23

4.7 Authentication and Route Protection 24

4.8 Screenshots of the System 25

4.9 Conclusion 34

CHAPTER 1

Introduction

1.1 Project Overview

StayEase is a web-based **Online Reservation and Booking System** for a boutique city hotel in Dhanmondi, Dhaka. Guests can browse room categories, see live availability and the total price for their dates, create an account and book a room in less than a minute. Hotel staff use a separate administration panel to manage room types, physical rooms, bookings, payments, customer accounts and guest reviews.

The system replaces phone calls, social-media messages and paper registers with one central, always-available platform. It is built as a full-stack web application with **Next.js 16**, **TypeScript** and **MongoDB**, and it is deployed on the internet at **stayease-murex.vercel.app**.

1.2 Background and Problem Statement

Many small and medium hotels in Bangladesh still take reservations manually. Requests arrive by phone or messenger and are written into a register or a spreadsheet. This way of working causes several recurring problems:

- **Double booking**: the same room is given to two guests for overlapping dates because there is no single source of truth.
- **No real-time availability**: a guest cannot see whether a room is free without contacting the hotel, and enquiries outside office hours are lost.
- **Unclear pricing**: the total cost of a stay is not known in advance.
- **Poor record keeping**: booking history, customer details and payment status are scattered and difficult to search.
- **High manual workload** for front-desk staff, and **no management overview** of arrivals, departures, occupancy or revenue.

StayEase solves these problems with an online system that shows live availability, calculates prices automatically, prevents overlapping bookings even when many guests book at the same moment, and gives the hotel one dashboard for all reservations.

1.3 Objectives

- Build a responsive web application where customers can search availability by date and book rooms online.
- Prevent double booking automatically using server-side validation and database transactions.
- Provide secure registration and login with role-based access for *Customers* and *Administrators*.
- Give administrators tools to manage room types, rooms, bookings, payments, users and reviews, with a statistics dashboard.
- Apply the software development life cycle in practice: requirement analysis, design, implementation, testing and deployment.

1.4 Scope of the Project

Table 1.1: Scope of the project

Included in the system	Not included (future work)
Public website: home, rooms, room details, availability search, about	Online payment gateway (bKash, SSLCommerz) — payment is recorded by the hotel
Registration, login, logout, profile and password change	E-mail / SMS notifications
Booking with automatic room assignment and price calculation	Multiple hotels (marketplace)
My bookings, cancellation and reviews after a completed stay	Seasonal or dynamic pricing
Admin dashboard, booking, room, customer and review management	Native mobile app (the website is fully responsive instead)

1.5 Target Users

Table 1.2: User roles

Role	Description	Main abilities
Guest (visitor)	Anyone who opens the website without logging in	Browse rooms, view details and reviews, check availability, register
Customer	A registered and logged-in guest	Book rooms, view and cancel bookings, write reviews, edit profile
Administrator	Hotel staff; accounts are created by a secure setup script	Dashboard, manage bookings & payments, rooms, room types, customers and reviews

1.6 Key Features

For customers

- Search availability by check-in date, check-out date and number of guests, with a date-range calendar.
- See how many rooms of each type are left and the total price for the stay.
- Book with a special request; receive a booking reference code (for example `SE-7K3P9Q`).
- Track bookings as *Upcoming*, *Past* and *Cancelled*, cancel free of charge until the day before arrival, and review a completed stay.

For administrators

- Dashboard with total and pending bookings, tonight's occupancy, paid revenue, and today's check-ins and check-outs.
- Search and filter bookings; confirm, cancel or complete them; record payment as paid or unpaid.
- Create and edit room types and rooms, put rooms under maintenance, block or unblock customers, and moderate reviews.

1.7 Organization of the Book

Chapter 2 describes the methodology: the development model, phases, requirements, tools and architecture. **Chapter 3** presents the design with flow charts, the entity-relationship diagram and data flow diagrams. **Chapter 4** explains the implementation with selected source code and screenshots of the working system.

CHAPTER 2

Methodology

2.1 Development Approach

The project followed an **incremental (iterative) development model**. Instead of building everything at once, the system was divided into seven increments. Each increment went through a short cycle of planning, building, testing and review, and the next increment started only after the previous one was working and verified. This kept the system demonstrable at every stage and allowed problems to be found early.

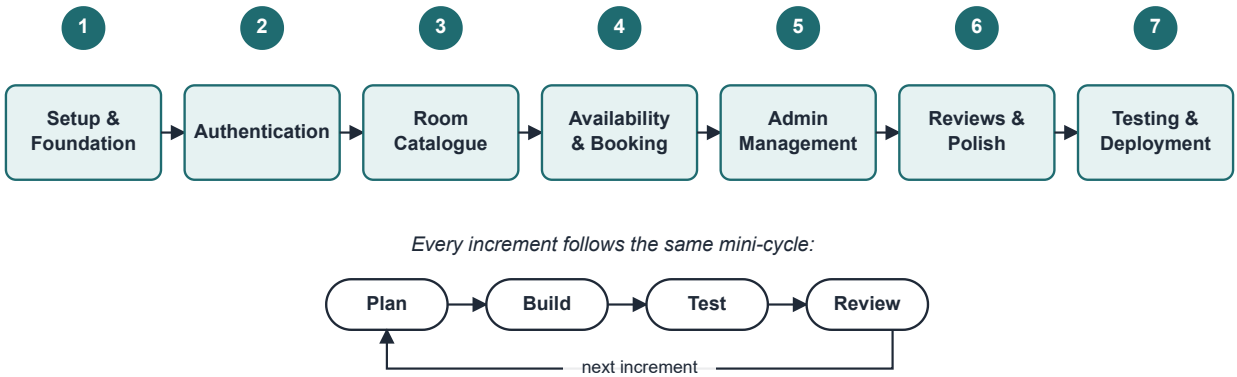


Figure 2.1: Incremental development model used for StayEase

2.2 Project Phases

Table 2.1: Development phases and outcomes

Phase	Work carried out	Outcome
1. Setup	Project initialisation, TypeScript, Tailwind CSS, database connection, folder structure, UI foundation	Running skeleton application with health check
2. Authentication	User model, registration, login, logout, password hashing, session cookie, route protection, admin setup script	Secure accounts with Customer and Admin roles
3. Room catalogue	Room type and room models, admin management pages, public room list and details	Hotel rooms can be managed and browsed
4. Availability & booking	Availability search, booking transaction with room lock, my bookings, cancellation	Customers can book without any double booking
5. Admin management	Dashboard statistics, booking and payment management, customer blocking	Complete back-office for hotel staff
6. Reviews & polish	Reviews and moderation, room filters, profile page, landing page design	Feature-complete, polished product
7. Testing & deployment	Automated test suite, demo data, MongoDB Atlas and Vercel deployment	System live on the internet

2.3 Requirement Analysis

Requirements were collected by studying how a small hotel takes reservations and were written as numbered functional requirements (for example `FR-BOOK-05`) so that every requirement could be traced to code and tests.

Table 2.2: Summary of functional requirements

Module	IDs	Main requirements
Authentication	FR-AUTH-01 – 09	Register, unique e-mail, bcrypt hashing, login, secure session, logout, role-based access, blocked users, profile & password
Room catalogue	FR-ROOM-01 – 03	List active room types, room details with reviews, filter by price and guests
Availability	FR-AVAIL-01 – 06	Search by dates and guests, date validation, maximum 30 nights, rooms left per type, overlap rule, maintenance rooms excluded
Booking	FR-BOOK-01 – 10	Customers only, summary before confirmation, server-side price, automatic room assignment, transaction with room lock, reference code, price snapshot, my bookings, cancellation, special request
Reviews	FR-REV-01 – 04	Review only after a completed stay, rating 1–5, average rating, moderation
Administration	FR-ADM-01 – 09	Dashboard, room type and room management, no deletion with active bookings, booking search, status changes, payment, user blocking, review moderation

Non-functional requirements included security (hashed passwords, HTTP-only cookies, server-side validation and authorisation), data integrity (no overlapping bookings), performance (pages load within about two to three seconds), usability and responsive design for mobile, tablet and desktop, accessibility (semantic HTML, keyboard support, reduced-motion support) and maintainability (modular TypeScript code and automated tests).

2.4 Tools and Technologies

Table 2.3: Technology stack

Layer	Technology	Reason for choice
Framework	Next.js 16 (App Router), React 19	Frontend and backend in one project; server components and server actions
Language	TypeScript (strict mode)	Type safety and self-documenting code
Styling / UI	Tailwind CSS 4, shadcn/ui (Radix), Lucide icons	Fast, consistent, accessible and responsive interface
Database	MongoDB 8 on MongoDB Atlas, Mongoose 9	Flexible document model; replica set provides multi-document transactions
Validation	Zod 4, React Hook Form	One schema validates both the browser form and the server
Security	jose (JWT), bcryptjs	Signed session tokens in HTTP-only cookies; password hashing
Testing	Vitest, mongodb-memory-server	Fast automated tests against a real in-memory MongoDB replica set
Deployment	Vercel, MongoDB Atlas, Git & GitHub	Free cloud hosting with automatic deployment on every push

2.5 System Architecture

StayEase uses a **three-tier architecture**. The browser renders the user interface and validates forms immediately. The Next.js server, hosted on Vercel, contains a proxy for fast route protection, server-rendered pages, server actions that handle form submissions, and a service layer with the business rules. The service layer uses Mongoose models to read and write data in MongoDB Atlas.

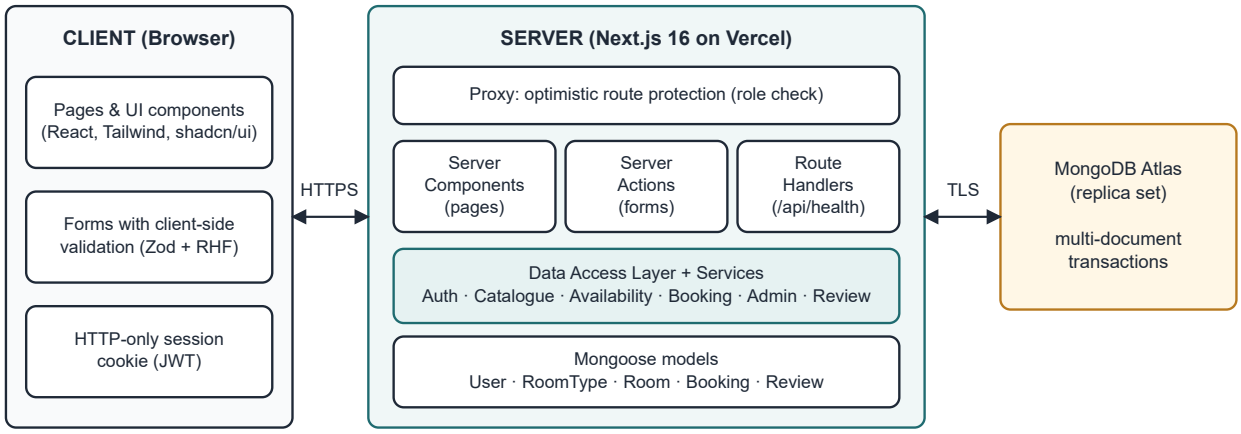


Figure 2.2: Three-tier architecture of StayEase

2.6 Key Technical Decisions

Preventing double booking

Every booking is created inside a **MongoDB multi-document transaction**. Inside the transaction the server finds the free rooms of the chosen type and then *locks* the first free room by incrementing its `lockVersion` field. If two customers try to book the same room at the same moment, MongoDB detects a write conflict on that room; the losing transaction is retried automatically on fresh data, sees the other booking and moves to the next free room or reports that no rooms are left. Two bookings can therefore never be stored for the same room and overlapping dates.

Availability rule

A room is free for the requested stay when its status is *available* and no *pending* or *confirmed* booking satisfies `existing.checkIn < requested.checkOut` and `existing.checkOut > requested.checkIn`. Cancelled and completed bookings never block a room, and the check-out day is not counted as a night, so one guest can leave on the same day another arrives.

Dates and security

Stay dates are handled as date-only values in the hotel's time zone (Asia/Dhaka) to avoid time-zone errors. Passwords are stored only as bcrypt hashes; the session is a signed token in an HTTP-only cookie; and every server action re-checks the user's role in the database, so blocking a user takes effect immediately.

2.7 Testing Approach

Testing was carried out at the end of every increment. **Automated tests** (Vitest) run against a real, temporary MongoDB replica set and cover validation rules, authentication, availability, the booking transaction (including twelve simultaneous booking requests), cancellation, administration and reviews. **Manual tests** were performed in the browser on desktop and mobile screen sizes for every user flow.

Table 2.4: Test results

Check	Result
Automated tests (11 test files)	159 / 159 passed
TypeScript type check and ESLint	No errors
Production build	Successful
Concurrency test: 12 simultaneous requests for 2 free rooms	Exactly 2 bookings created, on different rooms
Manual browser tests (desktop 1440 px, mobile 375 px)	All user and admin flows passed

2.8 Deployment

The database runs on a free **MongoDB Atlas** cluster (AWS, Mumbai region, the closest to Dhaka). The application is deployed on **Vercel**, which builds and publishes a new version automatically whenever code is pushed to the GitHub repository. Secrets such as the database connection string and the token secret are stored as environment variables and never in the source code.

CHAPTER 3

Design

This chapter presents the design of StayEase using three standard models: **flow charts** describe the step-by-step processes, the **entity-relationship (E-R) diagram** describes the stored data and its relationships, and **data flow diagrams (DFD)** describe how data moves between users, processes and data stores.

3.1 Flow Chart

3.1.1 Customer booking process

Figure 3.1 shows how a customer moves from opening the website to a confirmed booking. Two checks protect the process: the dates are validated before the search (in the browser and again on the server), and the final availability check is done inside a database transaction just before the booking is saved.

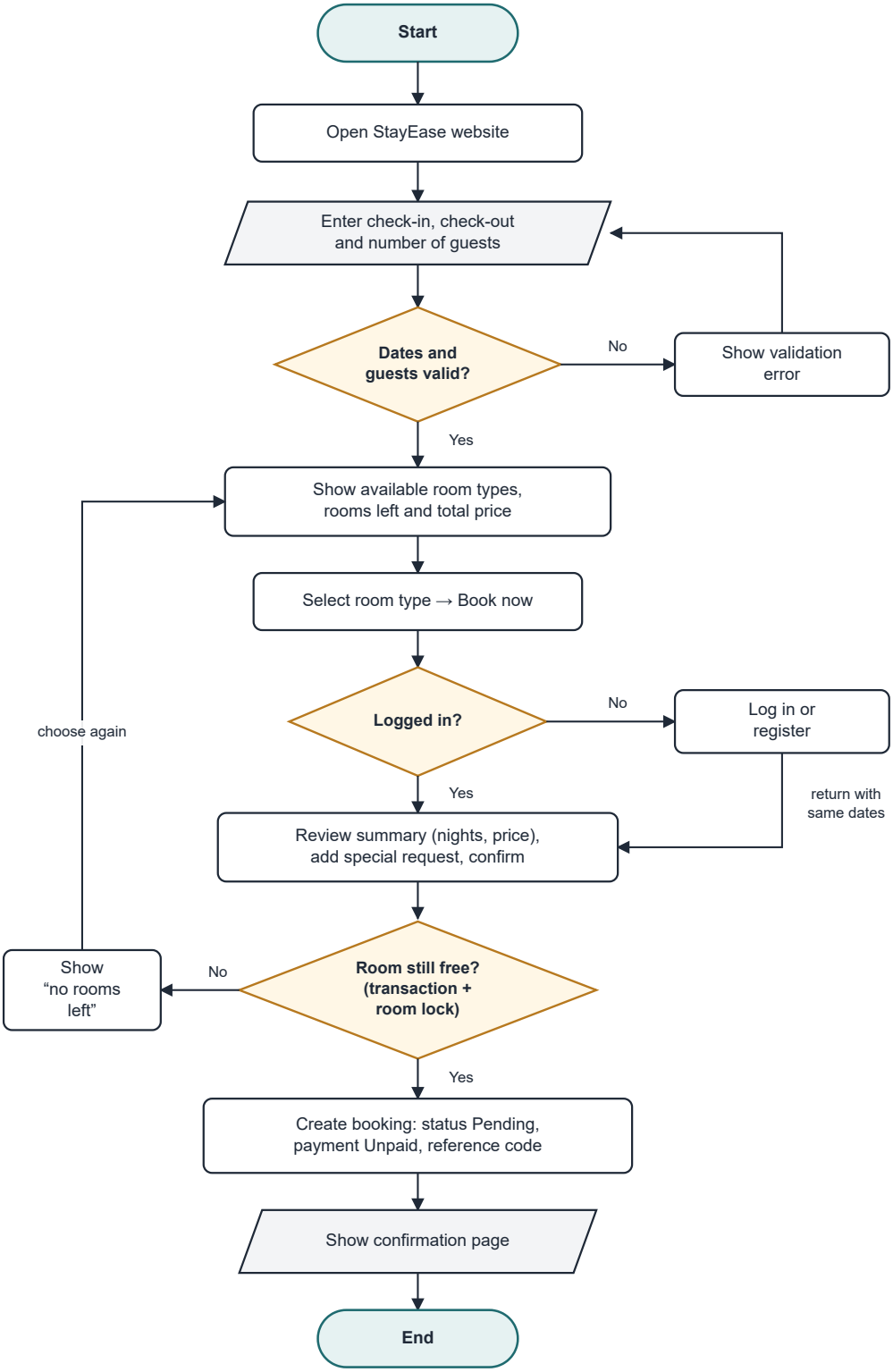


Figure 3.1: Flow chart of the customer booking process

3.1.2 Administrator booking management

Figure 3.2 shows how hotel staff handle a booking. A pending booking is confirmed or cancelled; a confirmed booking is marked completed after the guest's stay; and payment is recorded as paid when the guest pays at the hotel.

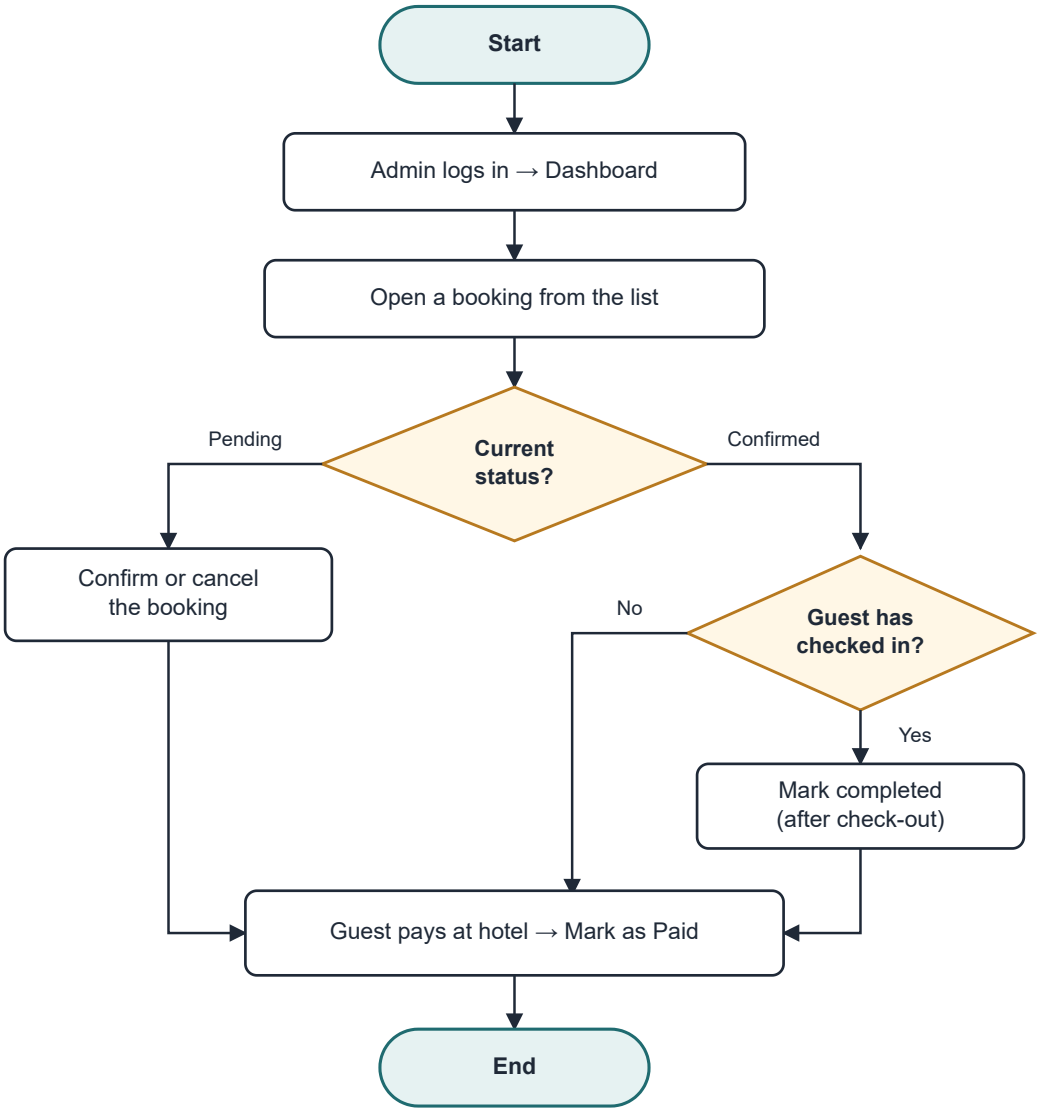


Figure 3.2: Flow chart of booking management by the administrator

3.2 E-R Diagram

StayEase stores its data in five collections (entities). A **ROOM_TYPE** such as “Deluxe Double Room” describes a kind of room and its price, while a **ROOM** is a physical room (for example No. 203) of one room type. A **BOOKING** links one **USER** to one assigned ROOM for a date range, and keeps a copy of the price at booking time. A **REVIEW** belongs to one completed booking.

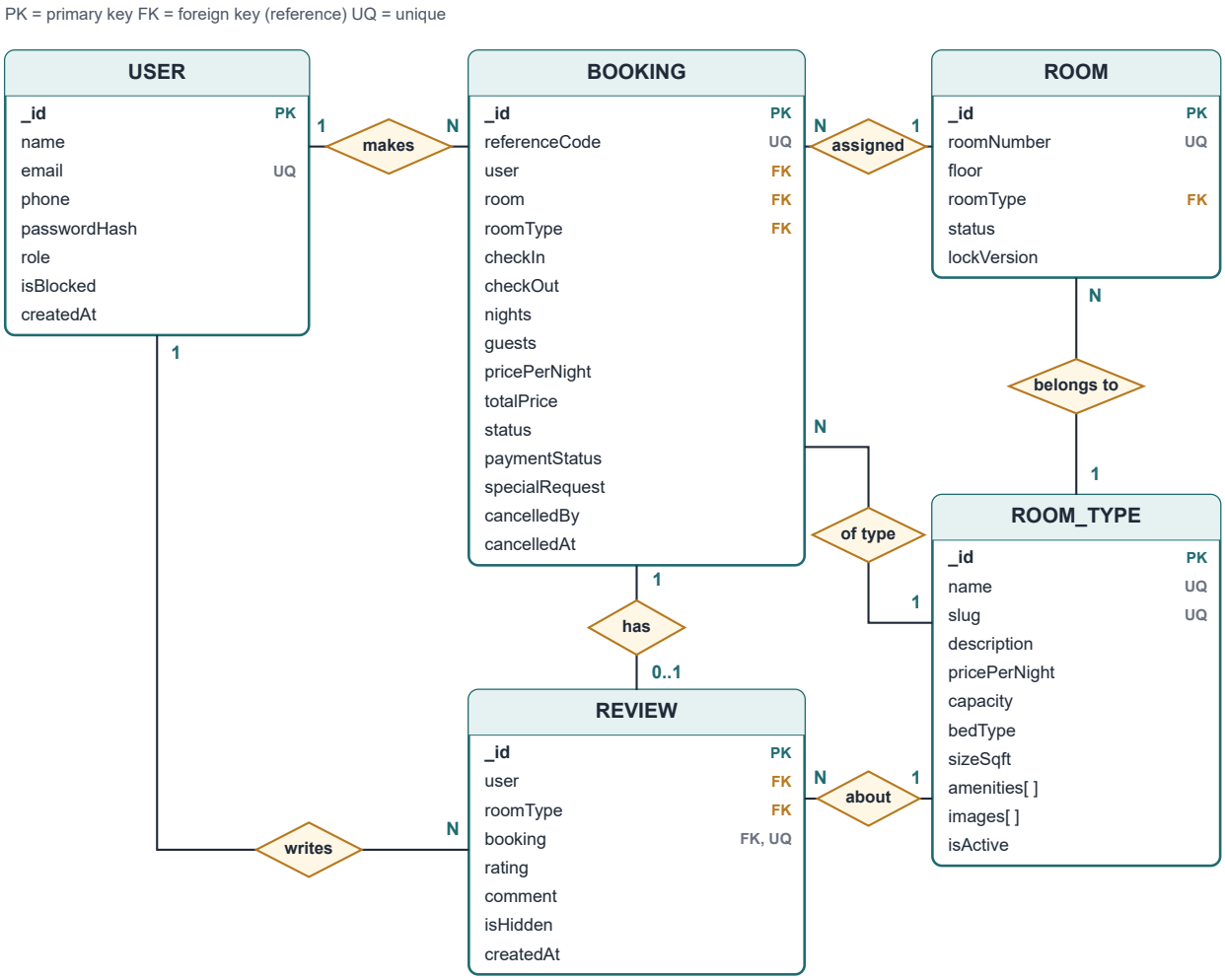


Figure 3.3: Entity-relationship diagram of StayEase

Table 3.1: Relationships between entities

Relationship	Cardinality	Meaning
USER makes BOOKING	1 : N	A customer can make many bookings; each booking belongs to one customer
BOOKING assigned ROOM	N : 1	Each booking occupies exactly one physical room
ROOM belongs to ROOM_TYPE	N : 1	A room type has many rooms (e.g. rooms 201–206 are Deluxe Double)
BOOKING of type ROOM_TYPE	N : 1	Stored for fast queries and reviews
BOOKING has REVIEW	1 : 0..1	A completed booking can have at most one review
USER writes REVIEW	1 : N	A customer can review each completed stay
REVIEW about ROOM_TYPE	N : 1	Ratings are averaged per room type

Table 3.2: Important attributes

Entity	Attribute	Description
USER	role, isBlocked	customer or admin; blocked users cannot log in
ROOM	status, lockVersion	available / maintenance / inactive; lock counter used by the booking transaction
BOOKING	checkIn, checkOut, nights	Date-only stay; check-out day is not a night
BOOKING	pricePerNight, totalPrice	Price snapshot calculated on the server at booking time
BOOKING	status, paymentStatus	pending / confirmed / cancelled / completed; unpaid / paid
BOOKING	referenceCode	Unique code such as SE-7K3P9Q given to the guest
REVIEW	rating, isHidden	1–5 stars; hidden reviews are not shown publicly

3.3 Data Flow Diagram (DFD)

3.3.1 Level 0 – Context diagram

The context diagram shows the whole system as a single process and the two external entities that interact with it: the **Customer** and the **Administrator**.

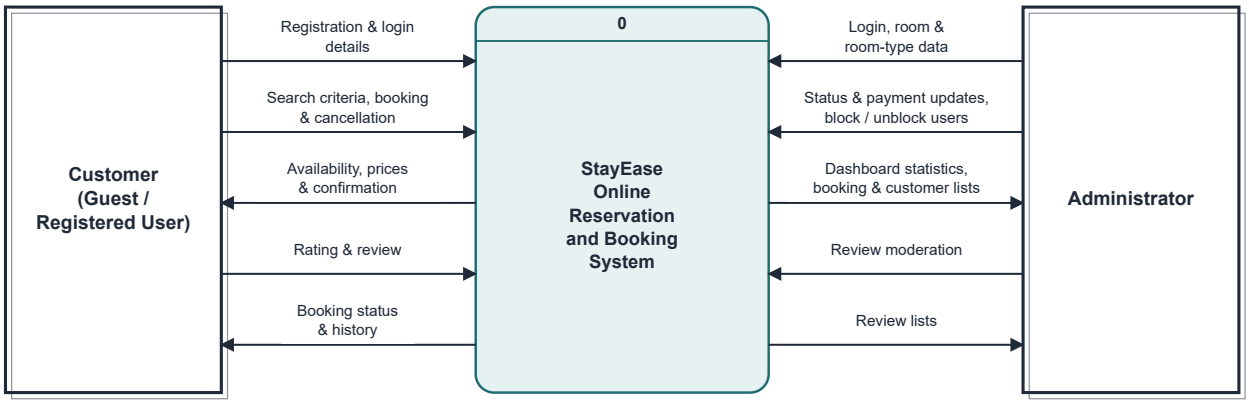


Figure 3.4: DFD Level 0 (context diagram)

3.3.2 Level 1 diagram

The Level 1 diagram breaks the system into seven processes and shows the five data stores (D1–D5) that correspond to the database collections.

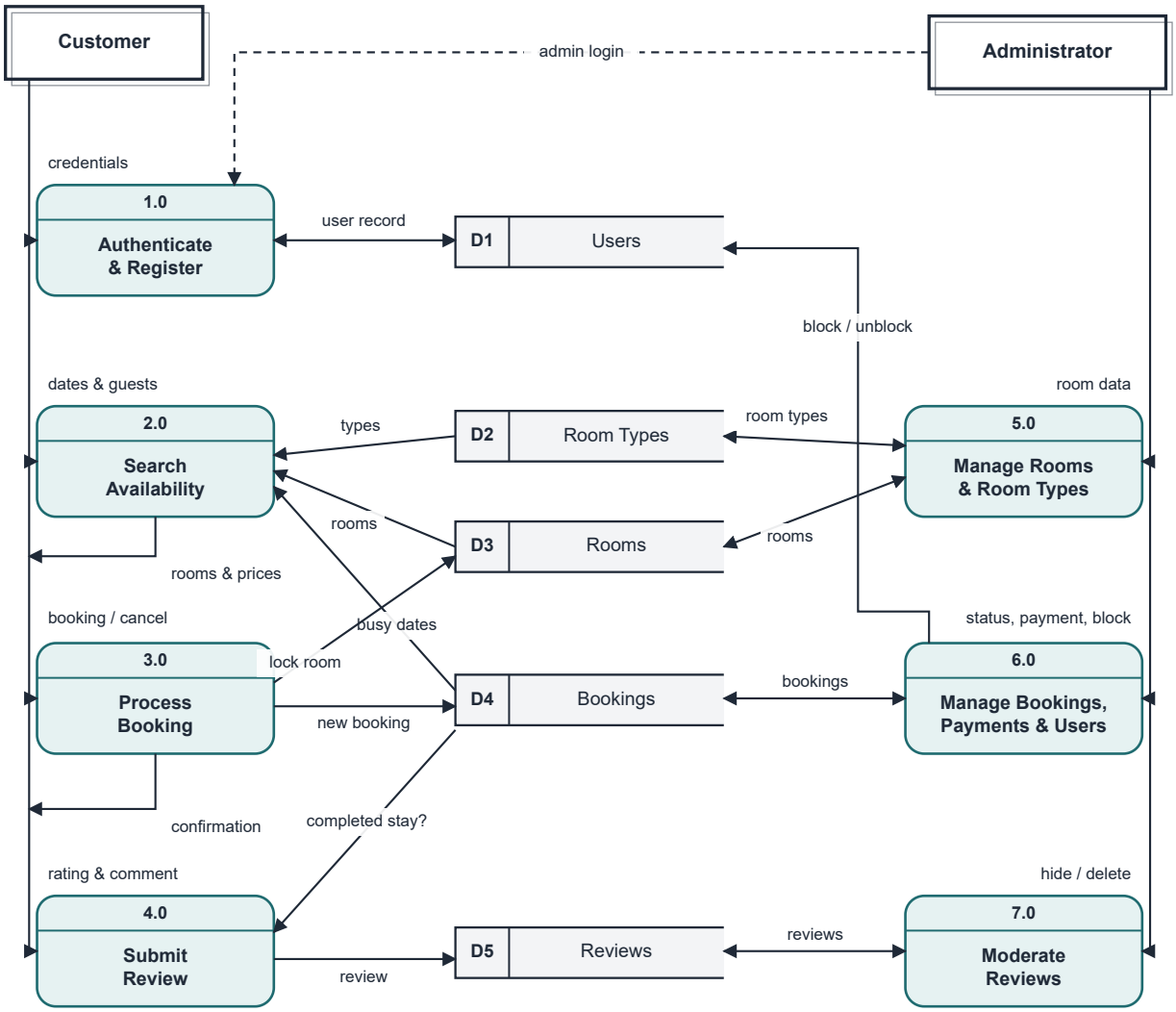


Figure 3.5: DFD Level 1

Table 3.3: Processes in the Level 1 DFD

No.	Process	Description
1.0	Authenticate & Register	Creates accounts, checks credentials and issues the session for customers and administrators (D1)
2.0	Search Availability	Reads room types, rooms and active bookings to calculate rooms left and total price (D2, D3, D4)
3.0	Process Booking	Locks a free room and stores the new booking in one transaction; handles cancellation (D3, D4)
4.0	Submit Review	Checks that the stay is completed and stores the rating and comment (D4, D5)
5.0	Manage Rooms & Room Types	Creates and updates room types and rooms, maintenance and deactivation (D2, D3)
6.0	Manage Bookings, Payments & Users	Confirms, cancels and completes bookings, records payment, blocks users, prepares the dashboard (D1, D4)
7.0	Moderate Reviews	Hides, shows or deletes reviews (D5)

CHAPTER 4

Implementation

This chapter explains how the design was implemented. Section 4.1 shows the structure of the source code, sections 4.2 to 4.7 present the most important parts of the code, and section 4.8 shows screenshots of the finished system. All listings are taken directly from the project's source files; the file name and line numbers are given above each listing.

4.1 Project Structure

The project is organised by responsibility. Pages only display data; all business rules live in the `services` folder, which makes them easy to test; and validation schemas are shared between the browser and the server.

```
stayease/
├── src/
│   ├── app/            Pages and routes (App Router)
│   │   ├── (home)/     Home page
│   │   ├── (site)/     Rooms, search, booking, my bookings, profile, about
│   │   ├── (auth)/     Login and registration
│   │   ├── admin/      Admin dashboard, bookings, rooms, customers, reviews
│   │   └── api/health  Health-check endpoint
│   ├── actions/        Server actions (form submissions)
│   ├── components/     UI components (booking, rooms, admin, layout, ui)
│   ├── lib/            Database connection, dates, authentication helpers
│   ├── models/         Mongoose models: User, RoomType, Room, Booking, Review
│   ├── services/       Business logic: availability, booking, admin, reviews
│   ├── validations/    Zod schemas shared by forms and server
│   └── proxy.ts        Route protection
├── scripts/            Database setup and demo-data scripts
├── tests/              Automated tests (Vitest)
└── docs/               Project plan and testing documents
```

Table 4.1: Size of the source code

Part	Approximate size
Application source (src/) and scripts	≈ 11,400 lines of TypeScript / TSX
Automated tests	11 test files, 159 test cases
Pages (routes)	20 pages + 1 API endpoint

4.2 Database Model

Listing 4.1 shows the Booking model. Dates are stored without time of day, the price per night is copied at booking time, and two indexes make availability checks and the “My bookings” list fast.

Listing 4.1: Booking model (Mongoose schema) `src/models/booking.ts, lines 4–33`

```
/**
 * Booking collection (docs/PROJECT_PLAN.md §12.2 `bookings`).
 * checkIn / checkOut are date-only values stored as UTC midnight.
 * pricePerNight is a snapshot taken at booking time (FR-BOOK-07).
 */
const bookingSchema = new Schema(
  {
    referenceCode: { type: String, required: true, unique: true },
    user: { type: Schema.Types.ObjectId, ref: "User", required: true },
    room: { type: Schema.Types.ObjectId, ref: "Room", required: true },
    roomType: { type: Schema.Types.ObjectId, ref: "RoomType", required: true, index: true },
    checkIn: { type: Date, required: true },
    checkOut: { type: Date, required: true },
    nights: { type: Number, required: true, min: 1 },
    guests: { type: Number, required: true, min: 1 },
    pricePerNight: { type: Number, required: true, min: 0 },
    totalPrice: { type: Number, required: true, min: 0 },
    status: { type: String, enum: BOOKING_STATUSES, default: "pending", required: true },
    paymentStatus: { type: String, enum: PAYMENT_STATUSES, default: "unpaid", required: true },
    specialRequest: { type: String, default: "", maxlength: 500 },
    cancelledBy: { type: String, enum: ["customer", "admin", null], default: null },
    cancelledAt: { type: Date, default: null },
  },
  { timestamps: true },
);

// Fast overlap checks for one room (§12.4).
bookingSchema.index({ room: 1, checkIn: 1, checkOut: 1 });
// "My bookings" list.
bookingSchema.index({ user: 1, checkIn: -1 });
```

4.3 Availability Check

Listing 4.2 implements the overlap rule from Section 2.6. `findFreeRooms` returns the rooms of a room type that are available and not booked for the requested dates, in natural room-number order.

Listing 4.2: Overlap rule and free-room search `src/services/availability.service.ts, lines 24–64`

```
export function overlappingActiveBookingFilter(range: StayRange) {
  return {
    status: { $in: ACTIVE_BOOKING_STATUSES },
    checkIn: { $lt: parseDateString(range.checkOut) },
    checkOut: { $gt: parseDateString(range.checkIn) },
  };
}

/** Ids (as strings) of the given rooms that are booked during the range. */
export async function findBusyRoomIds(
  roomIds: Types.ObjectId[],
  range: StayRange,
```

```
  session?: ClientSession,
): Promise<Set<string>> {
  if (roomIds.length === 0) return new Set();
  const busy = await Booking.distinct(
    "room",
    { room: { $in: roomIds }, ...overlappingActiveBookingFilter(range) },
    { session },
  );
  return new Set(busy.map((id) => String(id)));
}

/**
 * Bookable rooms of one room type for the range, in natural room-number order.
 * Used by the booking transaction (pass its session so reads use its snapshot).
 */
export async function findFreeRooms(
  roomTypeId: Types.ObjectId | string,
  range: StayRange,
  session?: ClientSession,
) {
  const rooms = await Room.find({ roomType: roomTypeId, status: "available" })
    .select("_id roomNumber")
    .session(session ?? null)
    .lean();
  const busy = await findBusyRoomIds(rooms.map((r) => r._id), range, session);
  return rooms
    .filter((r) => !busy.has(String(r._id)))
    .sort((a, b) => a.roomNumber.localeCompare(b.roomNumber, "en", { numeric: true }));
}
```

4.4 Booking Transaction (No Double Booking)

Listing 4.3 is a helper that runs a function inside a MongoDB transaction and retries it automatically after a write conflict. Listing 4.4 uses it to create a booking: it locks the chosen room by incrementing `lockVersion` before inserting the booking, so concurrent requests for the same room are serialised by the database.

Listing 4.3: Transaction helper with automatic retry `src/lib/db-core.ts, lines 61–94`

```
/**
 * Runs `fn` inside a MongoDB multi-document transaction.
 *
 * `session.withTransaction` automatically retries the whole callback on
 * `TransientTransactionError` (e.g. a write conflict caused by two concurrent
 * bookings touching the same room) and retries the commit on
 * `UnknownTransactionCommitResult`. This is the foundation of the
 * double-booking protection described in docs/PROJECT_PLAN.md §12.5.
 *
 * Requires a replica set (MongoDB Atlas, or a local replica set).
 * The callback may run more than once, so it must not have side effects
 * outside the database.
 */
export async function withTransaction<T>(
  fn: (session: ClientSession) => Promise<T>,
): Promise<T> {
  const session = await mongoose.startSession();
  try {
```

```
    let result!: T;
    await session.withTransaction(
      async () => {
        result = await fn(session);
      },
      {
        readConcern: { level: "snapshot" },
        writeConcern: { w: "majority" },
        readPreference: "primary",
      },
    );
    return result;
  } finally {
    await session.endSession();
  }
}
```

Listing 4.4: Creating a booking with a room-level lock `src/services/booking.service.ts, lines 107–187`

```
/**
 * Creates a booking inside a MongoDB transaction with a room-level lock
 * (docs/PROJECT_PLAN.md §12.5, FR-BOOK-05):
 *
 *   1. Read the room type (must be active and large enough).
 *   2. Find the rooms of that type that are free for the dates.
 *   3. LOCK the first free room by incrementing its `lockVersion`
 *      (only if it is still "available").
 *   4. Insert the booking (pending, unpaid, price snapshot).
 *
 * If another transaction locked/booked the same room after our snapshot was
 * taken, step 3 fails with a WriteConflict. `withTransaction` then retries
 * the whole callback on a fresh snapshot, where the other booking is visible
 * and the room is skipped. Two bookings can therefore never commit for the
 * same room and overlapping dates.
 */
export async function createBooking(params: CreateBookingParams): Promise<CreateBookingResult> {
  const range = { checkIn: params.checkIn, checkOut: params.checkOut };
  const nights = nightsBetween(params.checkIn, params.checkOut);

  // A reference-code collision is astronomically unlikely, but retry if it happens.
  for (let attempt = 1; ; attempt++) {
    try {
      const bookingId = await withTransaction(async (session) => {
        const roomType = await RoomType.findOne({ slug: params.roomTypeSlug, isActive: true })
          .session(session)
          .lean();
        if (!roomType) throw new BookingFailure("room_type_unavailable");
        if (params.guests > roomType.capacity) throw new BookingFailure("too_many_guests");

        const freeRooms = await findFreeRooms(roomType._id, range, session);

        for (const candidate of freeRooms) {
          // Room-level lock (§12.5). Matches only if the room is still available.
          const lock = await Room.updateOne(
            { _id: candidate._id, status: "available" },
            { $inc: { lockVersion: 1 } },
            { session },
          );
          if (lock.matchedCount === 0) continue;
```

```
          const [booking] = await Booking.create(
            [
              {
                referenceCode: generateReferenceCode(),
                user: params.userId,
                room: candidate._id,
                roomType: roomType._id,
                checkIn: parseDateString(params.checkIn),
                checkOut: parseDateString(params.checkOut),
                nights,
                guests: params.guests,
                pricePerNight: roomType.pricePerNight, // snapshot (FR-BOOK-07)
                totalPrice: nights * roomType.pricePerNight, // server-side (FR-BOOK-03)
                status: "pending",
                paymentStatus: "unpaid",
                specialRequest: params.specialRequest,
              },
            ],
            { session },
          );
          return booking._id;
        }

        throw new BookingFailure("no_rooms_available");
      });

      const booking = await getBookingForUser(params.userId, String(bookingId));
      return { ok: true, booking: booking! };
    } catch (error) {
      if (error instanceof BookingFailure) {
        return { ok: false, reason: error.reason, message: error.message };
      }
      const isCodeCollision =
        (error as { code?: number; keyPattern?: object }).code === 11000 &&
        "referenceCode" in ((error as { keyPattern?: object }).keyPattern ?? {});
      if (isCodeCollision && attempt < 3) continue;
      throw error;
    }
  }
}
```

4.5 Input Validation

The same Zod rules validate the search form in the browser and every request on the server. Listing 4.5 checks that check-in is not in the past, that bookings are at most one year ahead, that check-out is after check-in and that a stay is at most 30 nights.

Listing 4.5: Stay date validation rules `src/validations/booking.ts, lines 39–64`

```
function refineStay(today: string) {
  return (data: StayShape, ctx: z.RefinementCtx) => {
    const { checkIn, checkOut } = data;
    if (!isValidDateString(checkIn) || !isValidDateString(checkOut)) return;

    if (checkIn < today) {
      ctx.addIssue({ code: "custom", path: ["checkIn"], message: "Check-in cannot be in the past"
});
    } else if (checkIn > addDays(today, MAX_DAYS_AHEAD)) {
```

```
      ctx.addIssue({
        code: "custom",
        path: ["checkIn"],
        message: `Bookings open at most ${MAX_DAYS_AHEAD} days in advance`,
      });
    }

    if (checkOut <= checkIn) {
      ctx.addIssue({ code: "custom", path: ["checkOut"], message: "Check-out must be after check-
in" });
    } else if (nightsBetween(checkIn, checkOut) > MAX_NIGHTS) {
      ctx.addIssue({
        code: "custom",
        path: ["checkOut"],
        message: `A stay can be at most ${MAX_NIGHTS} nights`,
      });
    }
  };
}
```

4.6 Server Action for Booking

Listing 4.6 is the server action called by the booking form. It checks the customer's role in the database, validates the input again, creates the booking and redirects to the confirmation page. The price is never taken from the browser.

Listing 4.6: Booking server action `src/actions/booking.ts, lines 26–51`

```
export async function createBookingAction(values: unknown): Promise<ActionFailure> {
  let bookingId: string;
  try {
    await dbConnect();
    const auth = await authorize("customer");
    if (!auth.ok) return { ok: false, formError: auth.message };

    const parsed = createBookingSchema().safeParse(values); // FR-AVAIL-02 (server side)
    if (!parsed.success) return validationFailure(parsed.error);

    const result = await createBooking({
      userId: auth.user.id,
      roomTypeSlug: parsed.data.roomType,
      checkIn: parsed.data.checkIn,
      checkOut: parsed.data.checkOut,
      guests: parsed.data.guests,
      specialRequest: parsed.data.specialRequest,
    });
    if (!result.ok) return { ok: false, formError: result.message };
    bookingId = result.booking.id;
  } catch (error) {
    console.error("[createBookingAction]", error);
    return UNEXPECTED_FAILURE;
  }
  redirect(`/booking/${bookingId}/confirmation`);
}
```

4.7 Authentication and Route Protection

After a successful login, Listing 4.7 stores a signed session token in an HTTP-only cookie that JavaScript in the browser cannot read. Listing 4.8 decides, for every protected page, whether to allow the request or redirect the user: guests are sent to the login page, customers cannot open admin pages and administrators are sent to their dashboard.

Listing 4.7: Creating the session cookie `src/lib/auth/session.ts, lines 22–37`

```
export async function createSession(user: SessionUser): Promise<void> {
  const { JWT_SECRET, JWT_EXPIRES_IN, NODE_ENV } = getEnv();
  const { token, expiresAt } = await signSessionToken(
    { userId: user.id, role: user.role },
    JWT_SECRET,
    JWT_EXPIRES_IN,
  );

  (await cookies()).set(SESSION_COOKIE_NAME, token, {
    httpOnly: true,
    secure: NODE_ENV === "production",
    sameSite: "lax",
    path: "/",
    expires: new Date(expiresAt * 1000),
  });
}
```

Listing 4.8: Role-based route protection `src/lib/auth/route-guard.ts, lines 15–40`

```
/**
 * Pure routing decision used by `proxy.ts` (optimistic check, FR-AUTH-07).
 * Kept separate from the proxy so it can be unit-tested.
 */
export function decideRoute(
  pathname: string,
  search: string,
  session: SessionPayload | null,
): GuardDecision {
  const isAdminArea = matchesPrefix(pathname, ADMIN_PATHS);
  const isCustomerArea = matchesPrefix(pathname, CUSTOMER_PATHS);
  const isAccountArea = matchesPrefix(pathname, ACCOUNT_PATHS);
  if (!isAdminArea && !isCustomerArea && !isAccountArea) return { action: "allow" };

  if (!session) {
    const next = encodeURIComponent(pathname + search);
    return { action: "redirect", to: `/login?next=${next}` };
  }
  if (isAdminArea && session.role !== "admin") {
    return { action: "redirect", to: CUSTOMER_HOME };
  }
  if (isCustomerArea && session.role !== "customer") {
    return { action: "redirect", to: ADMIN_HOME };
  }
  return { action: "allow" };
}
```

Automated test example

Listing 4.9 is one of the 159 automated tests. It sends twelve booking requests at the same moment for a room type that has only two free rooms and checks that exactly two bookings are stored, on two different rooms.

Listing 4.9: Concurrency test for double-booking prevention `tests/booking-service.test.ts, lines 167–178`

```
  it("12 simultaneous requests for 2 rooms → exactly 2 bookings, on different rooms", async () =>
{
    const results = await Promise.all(
      Array.from({ length: 12 }, (_, i) => book(i % 2 ? alice : bob, d(20), d(23))),
    );
    const ok = results.filter((r) => r.ok);
    expect(ok).toHaveLength(2);
    expect(results.filter((r) => !r.ok && r.reason === "no_rooms_available")).toHaveLength(10);

    const stored = await Booking.find({ status: "pending" }).lean();
    expect(stored).toHaveLength(2);
    expect(new Set(stored.map((b) => String(b.room))).size).toBe(2);
  });
```

4.8 Screenshots of the System

4.8.1 Public website

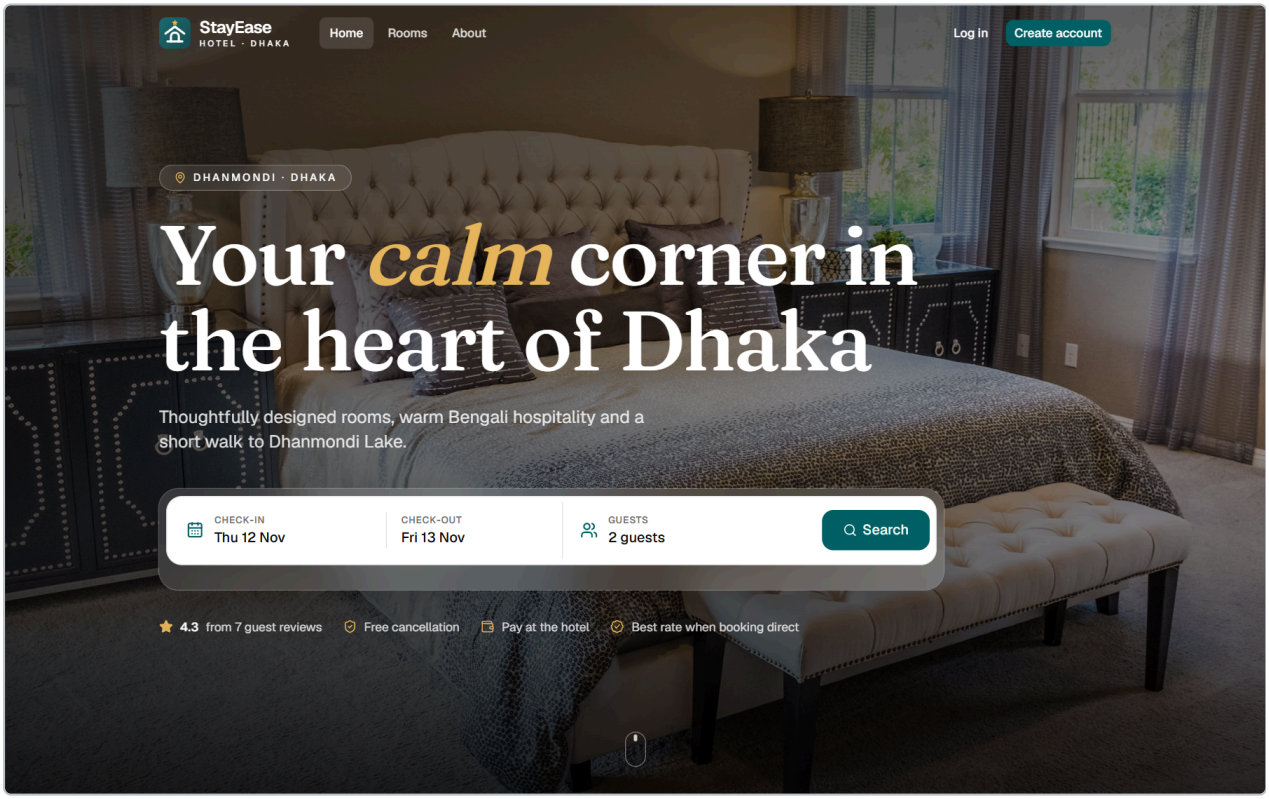


Figure 4.1: Home page with live availability search

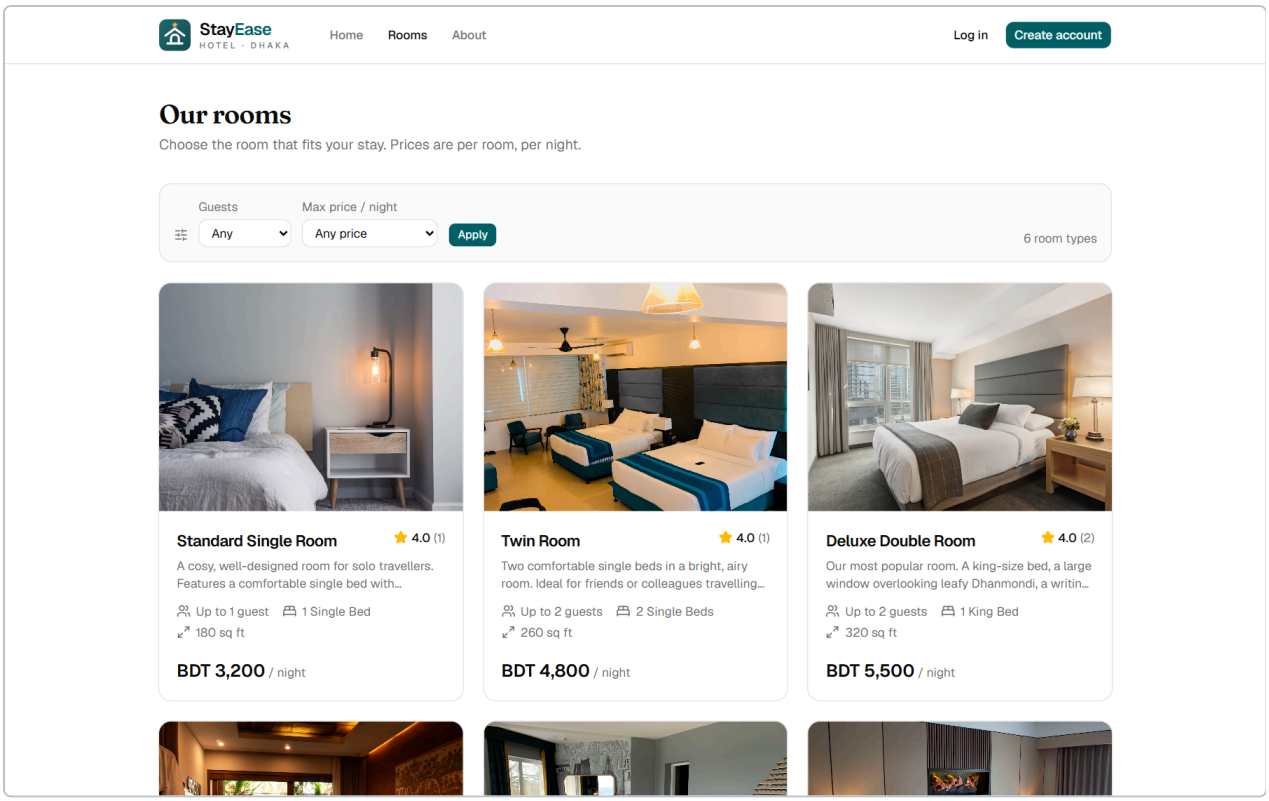


Figure 4.2: Room list with price and guest filters

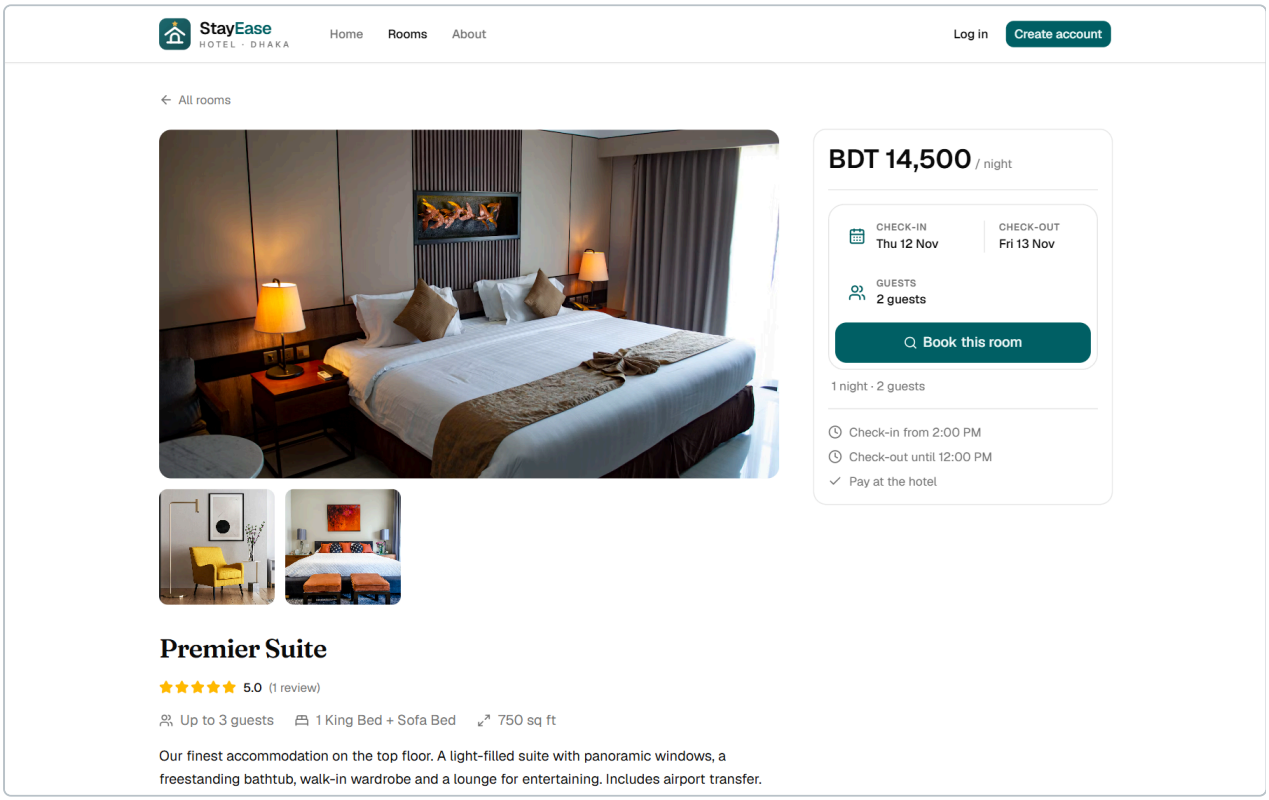


Figure 4.3: Room details page with booking form and reviews

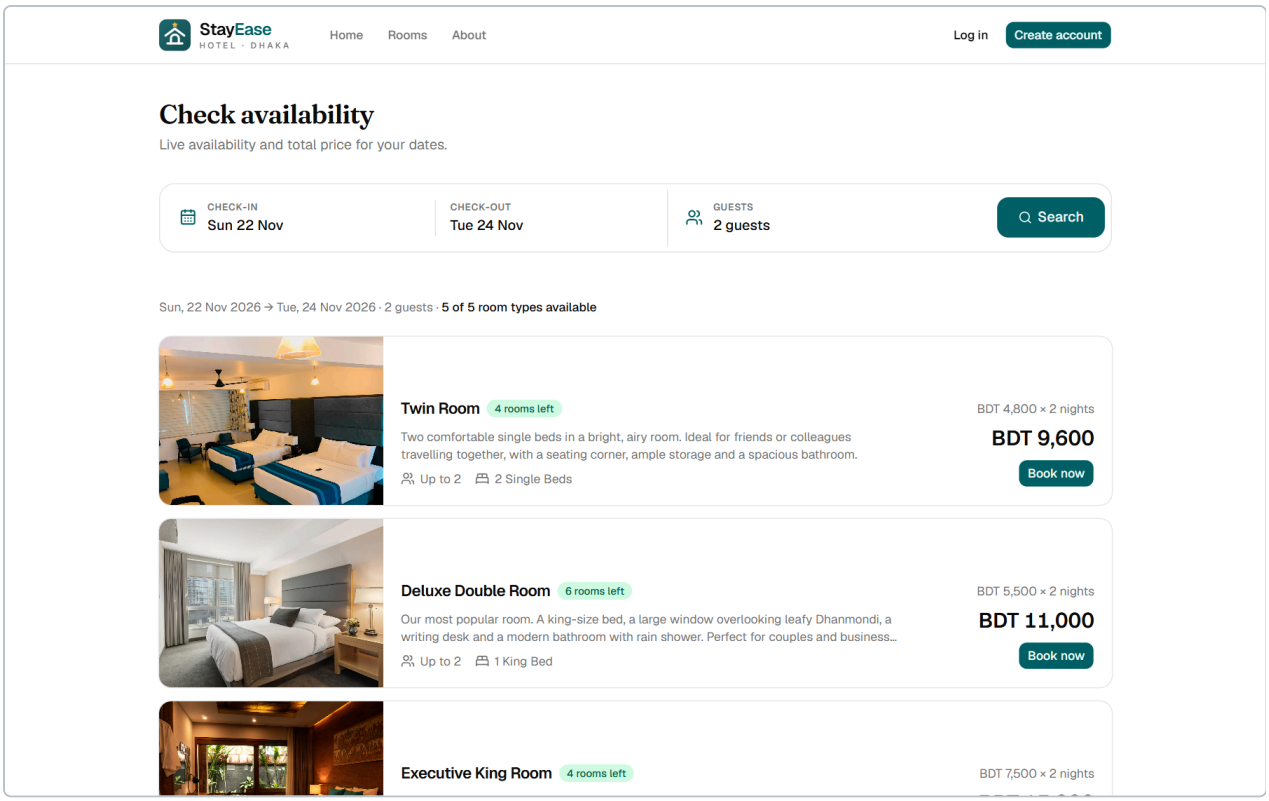


Figure 4.4: Availability search results with rooms left and total price

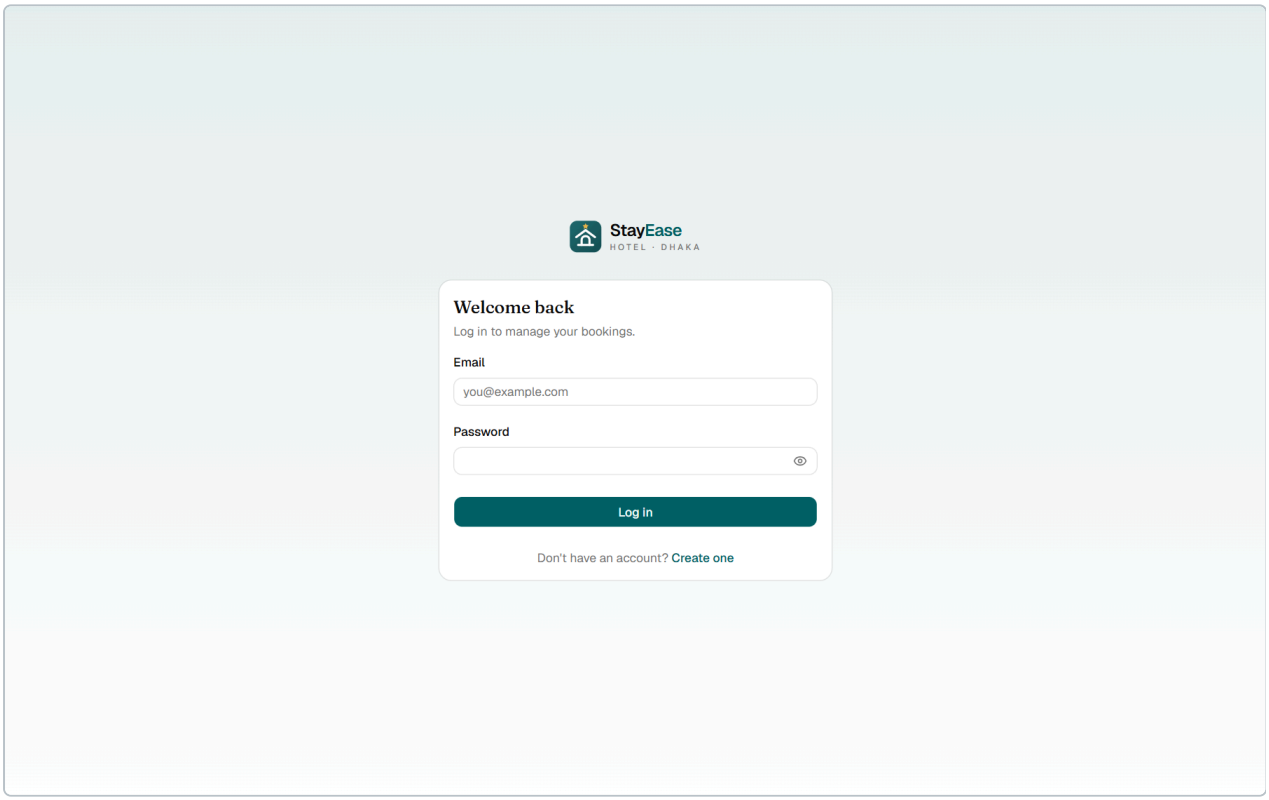


Figure 4.5: Login page

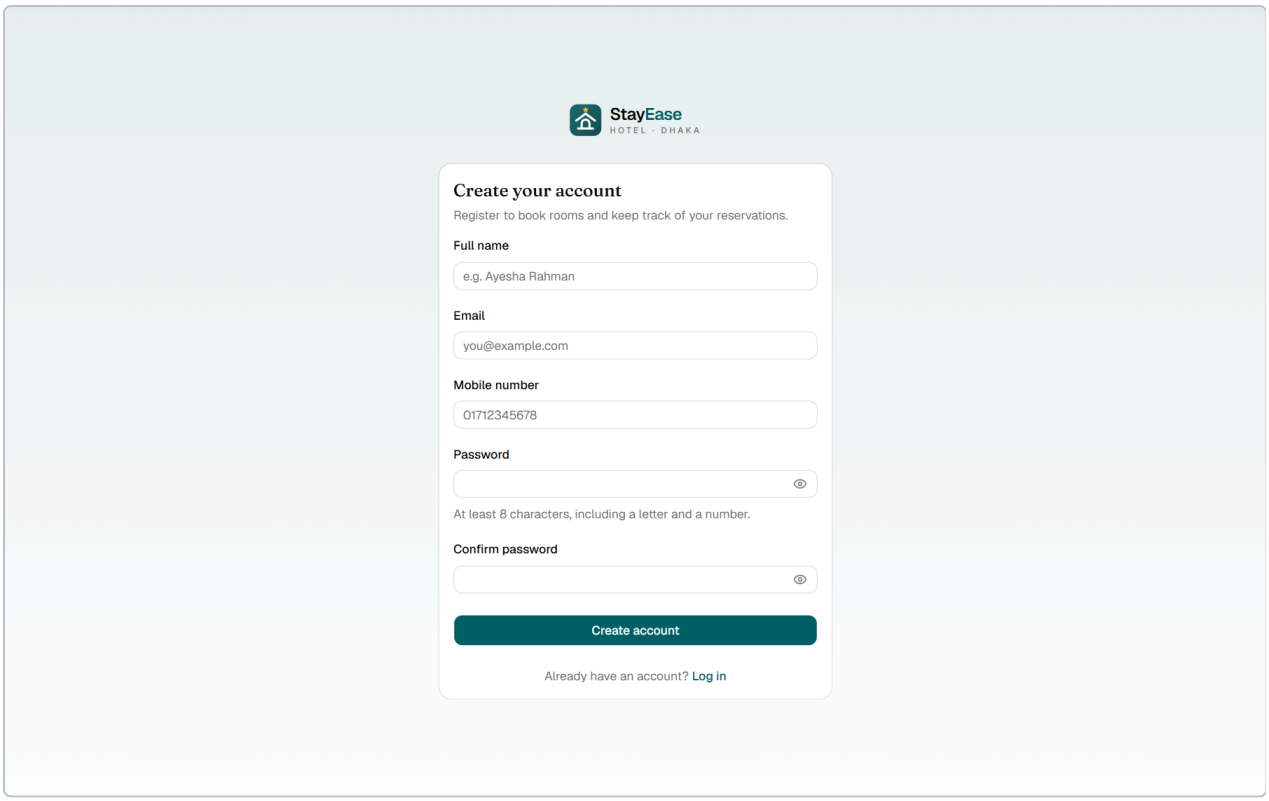


Figure 4.6: Registration page with validation

4.8.2 Customer pages

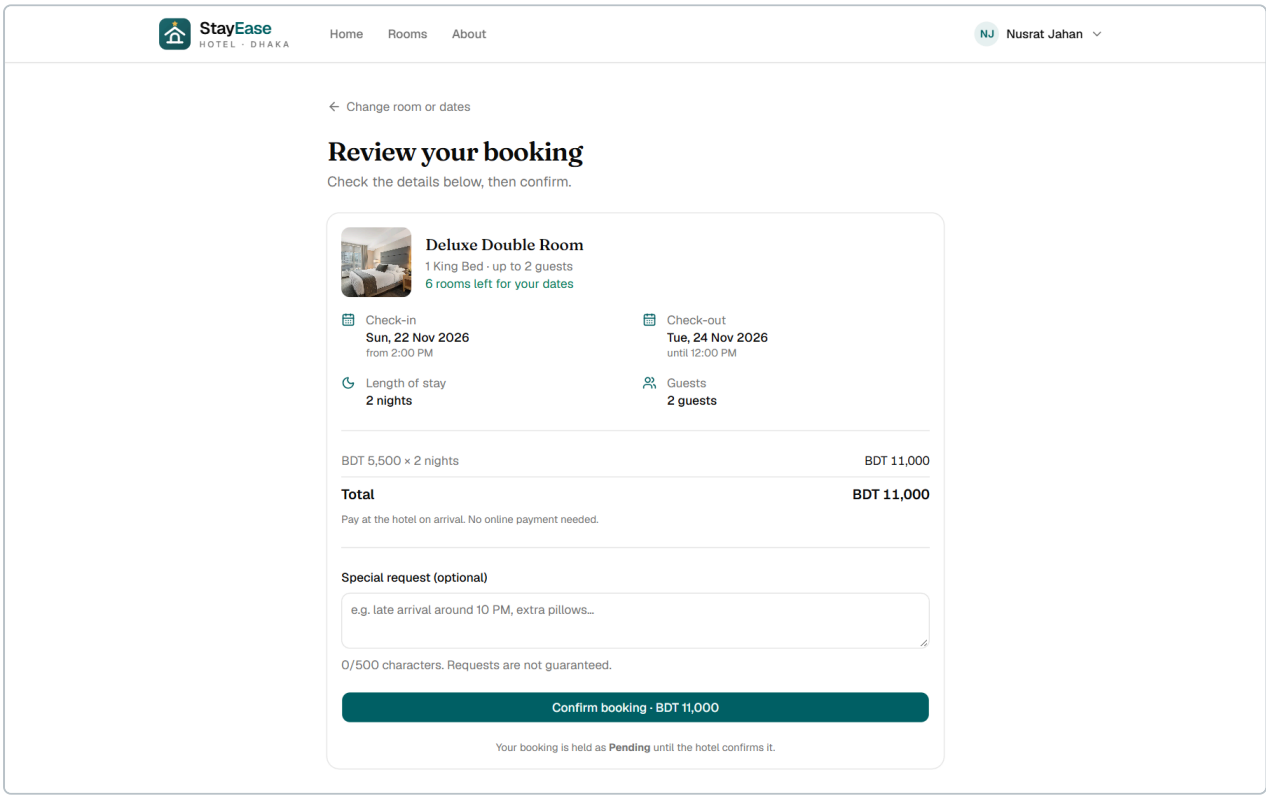


Figure 4.7: Booking review page before confirmation

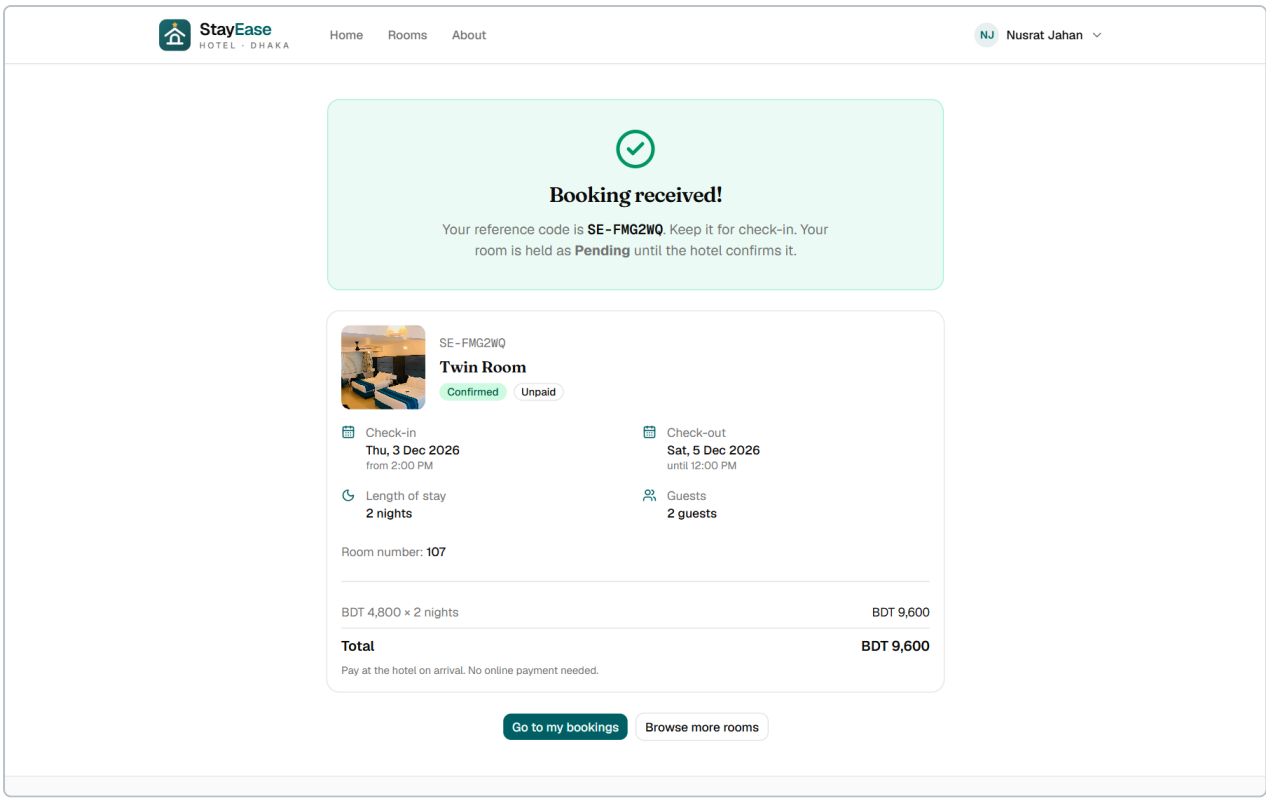


Figure 4.8: Booking confirmation with reference code

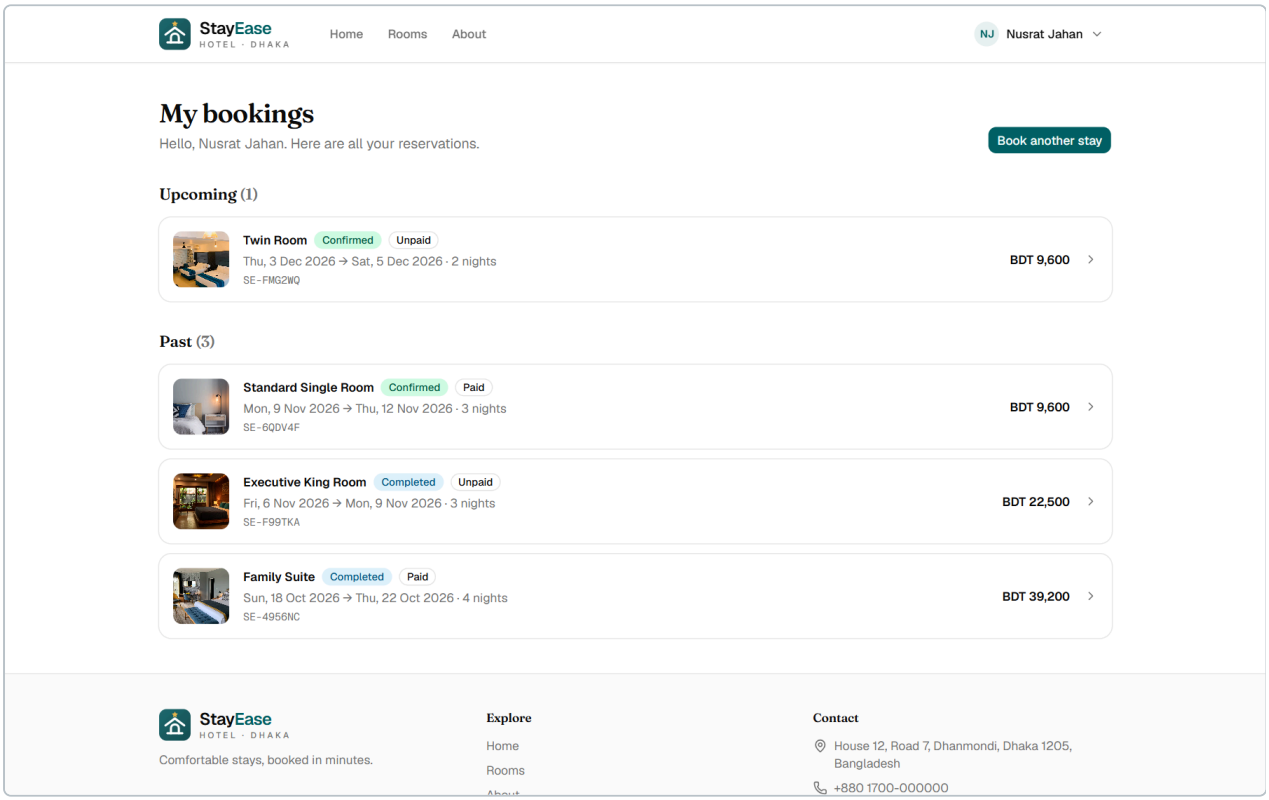


Figure 4.9: My bookings: upcoming, past and cancelled

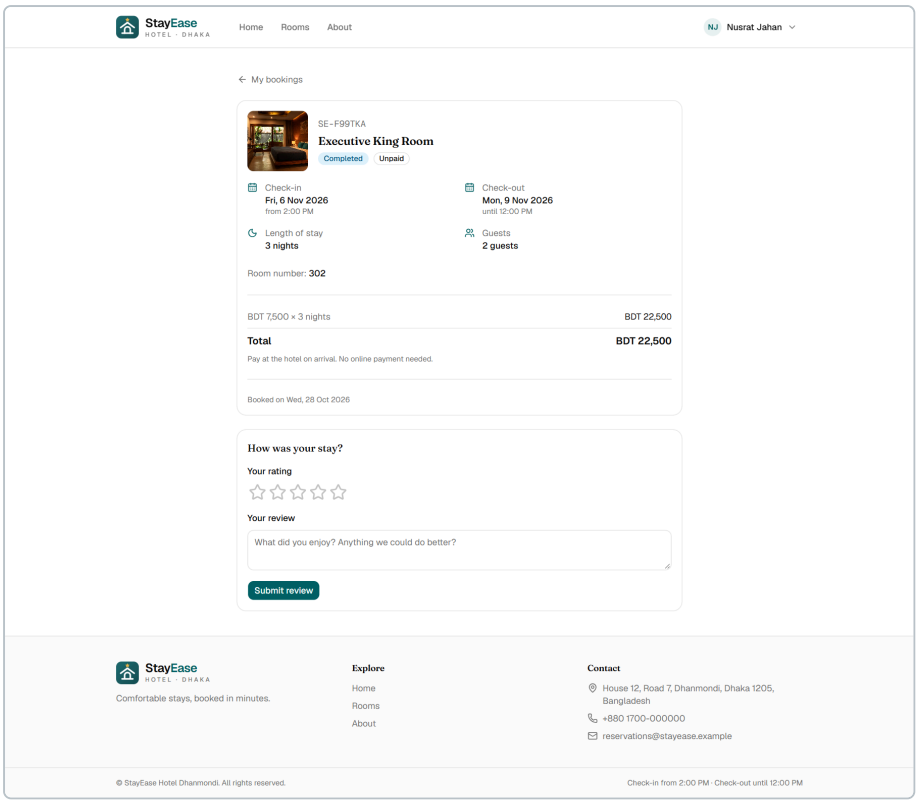


Figure 4.10: Completed stay with the review form

4.8.3 Administrator pages

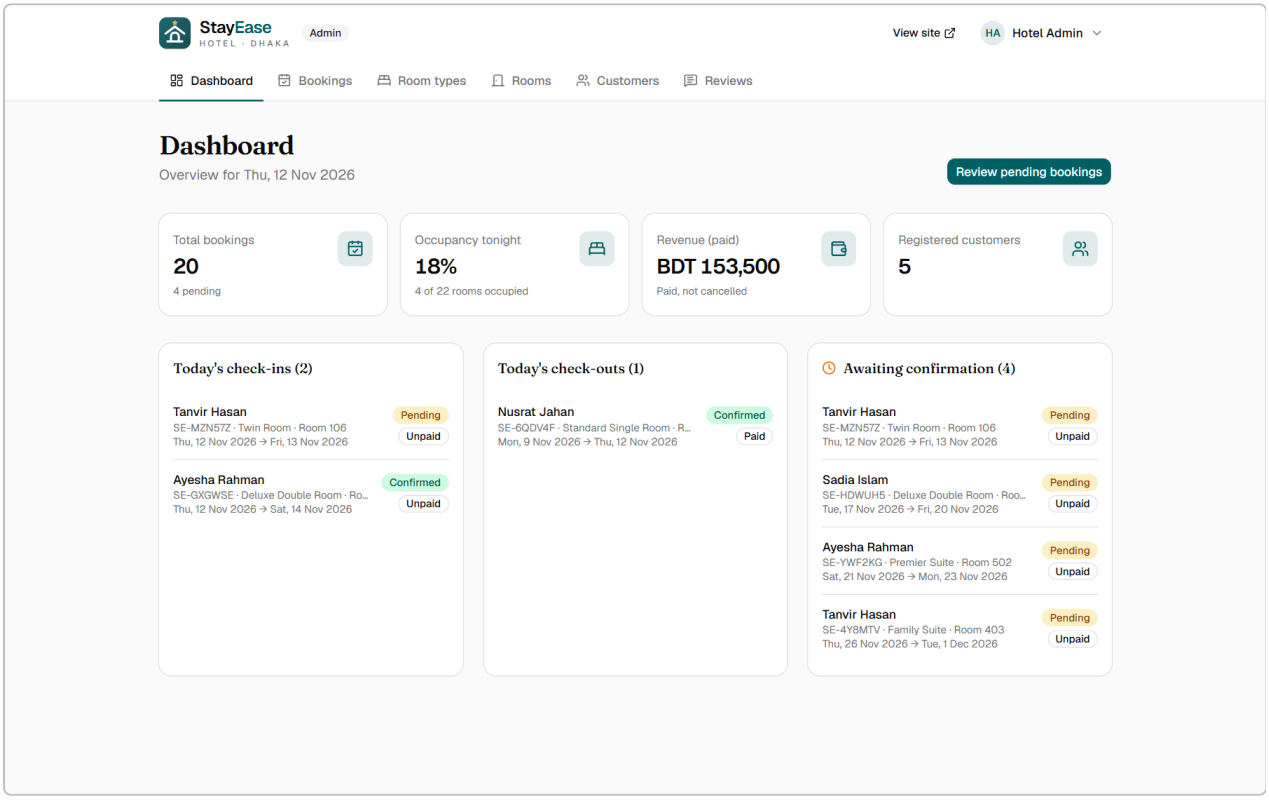


Figure 4.11: Admin dashboard with statistics and today's arrivals and departures

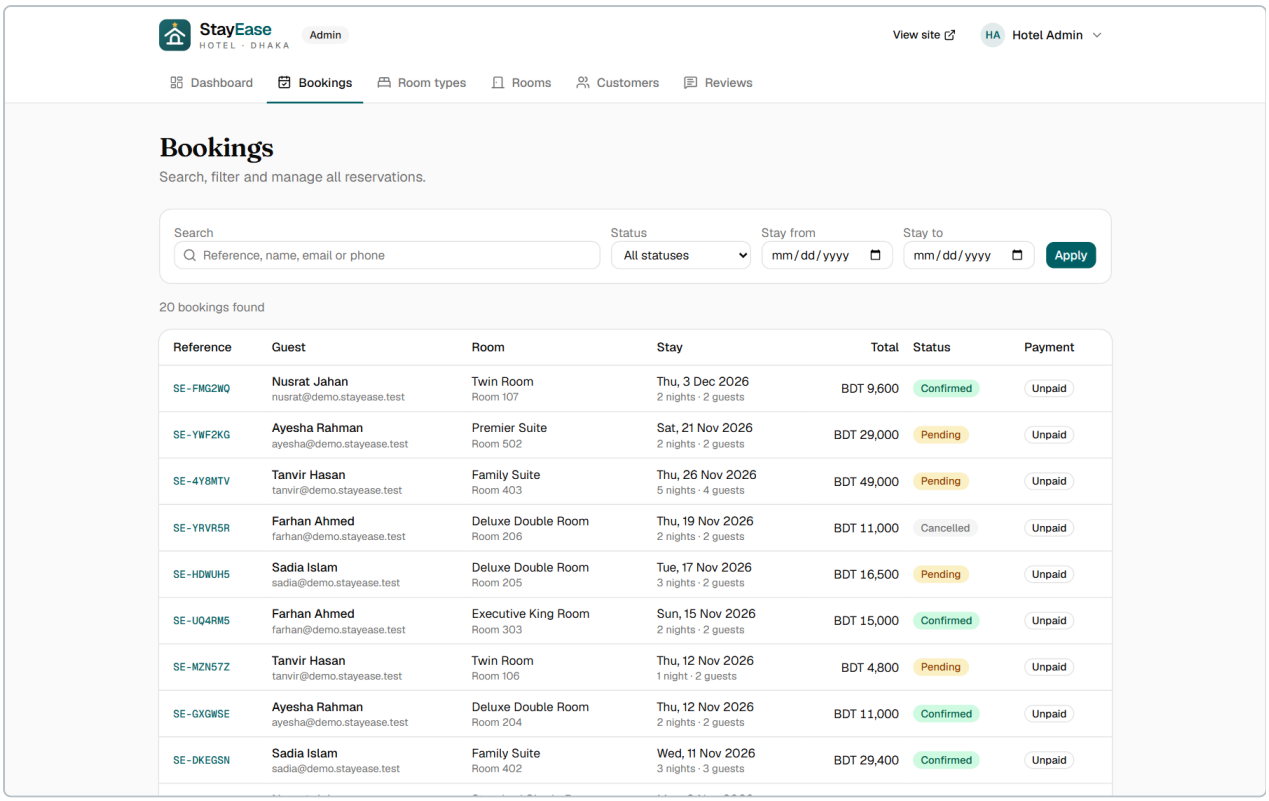


Figure 4.12: Booking management with search and filters

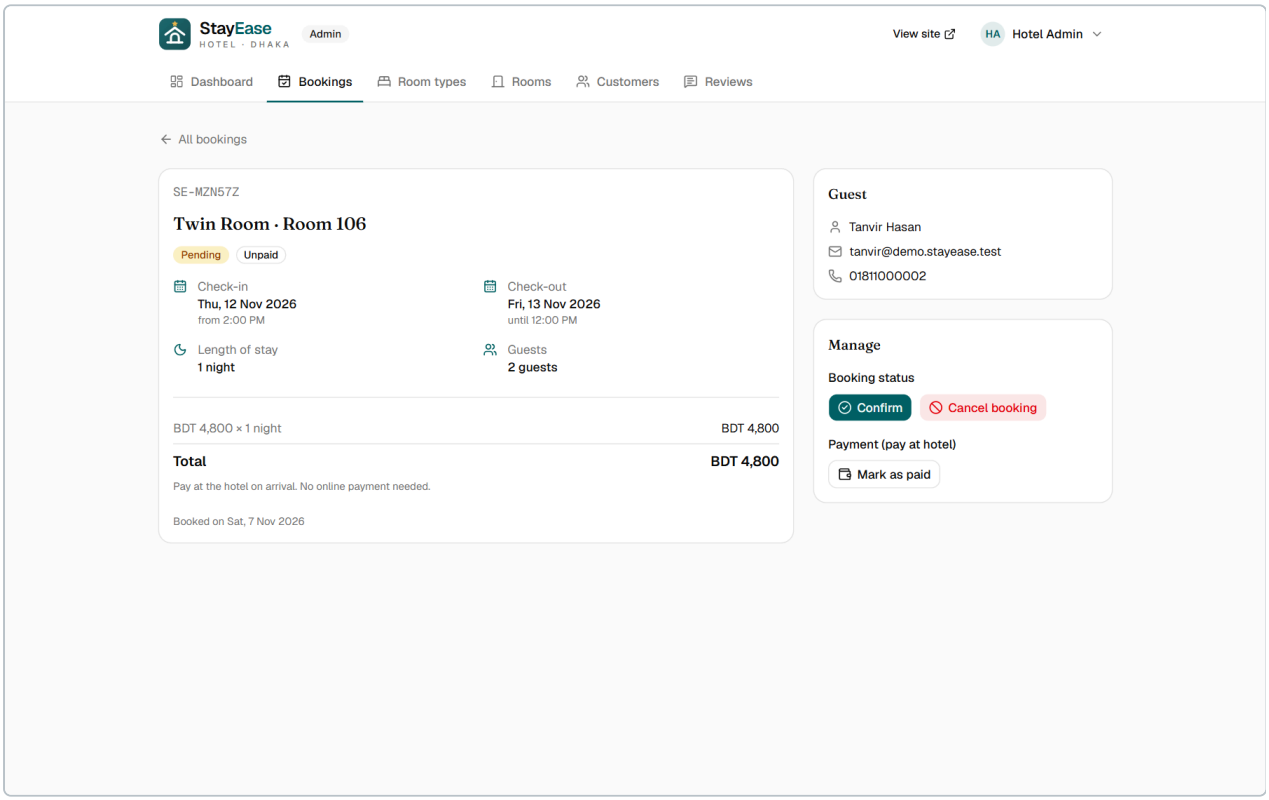


Figure 4.13: Booking details with status and payment controls

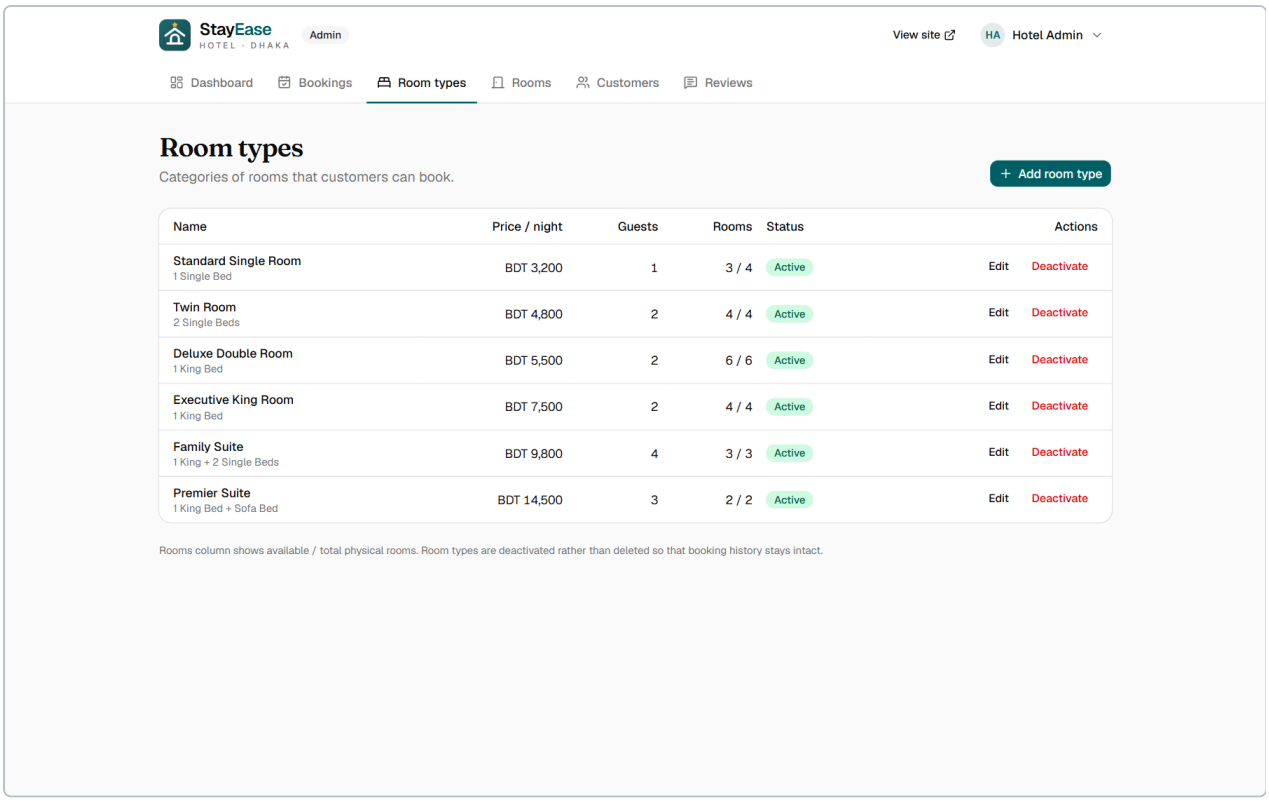


Figure 4.14: Room type management

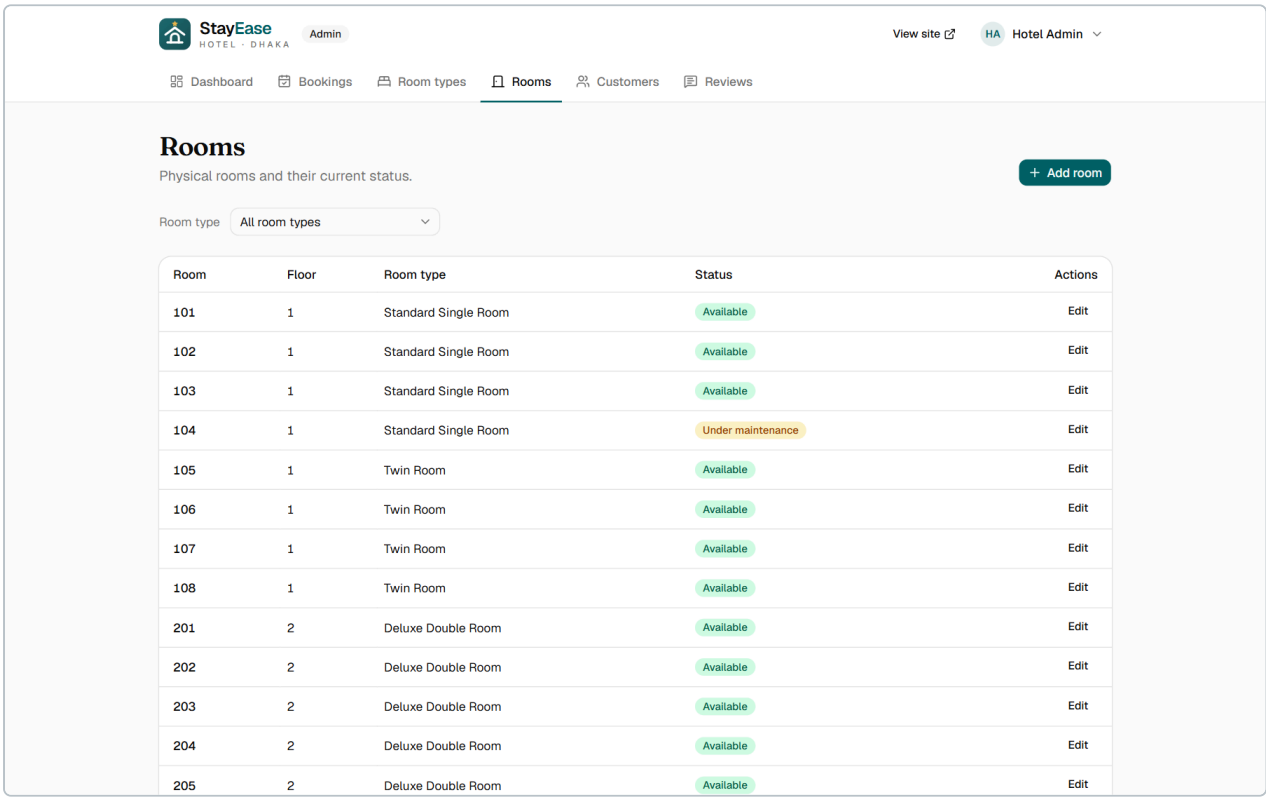


Figure 4.15: Physical room management

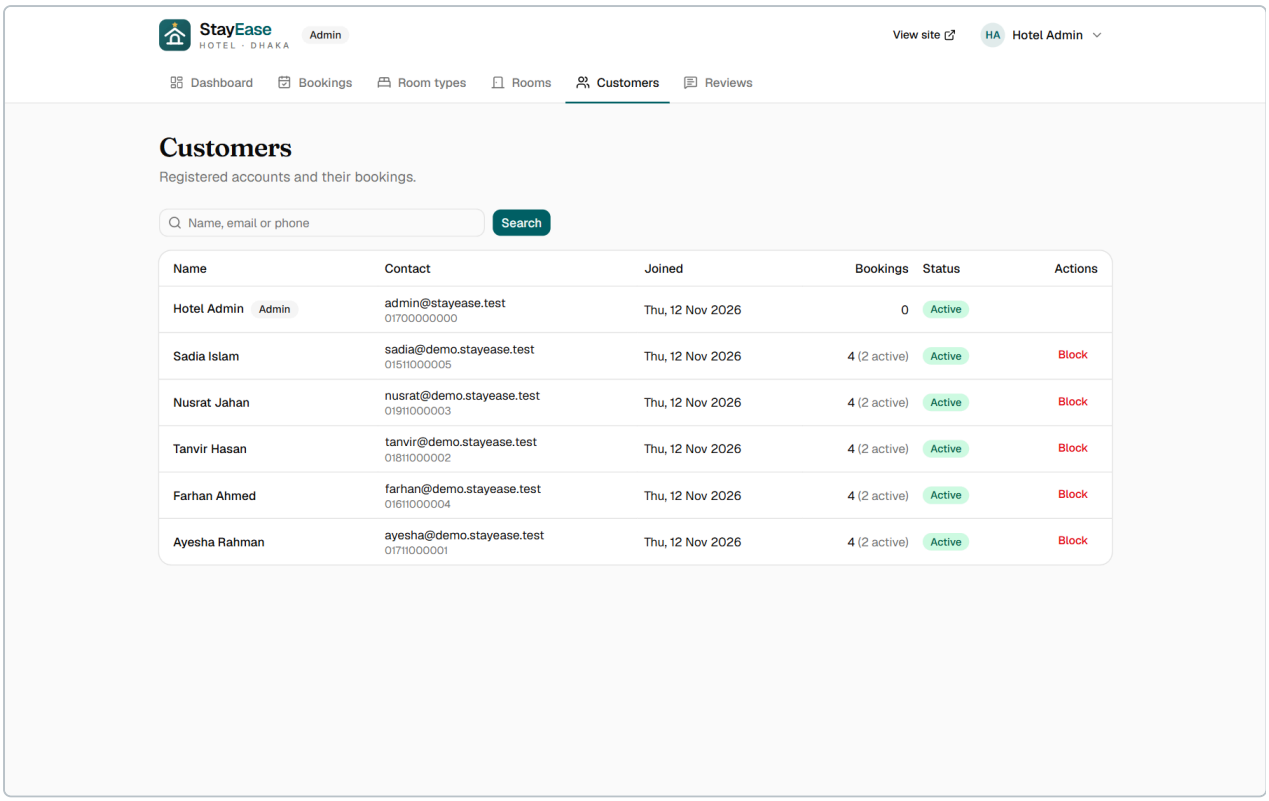


Figure 4.16: Customer management with block and unblock

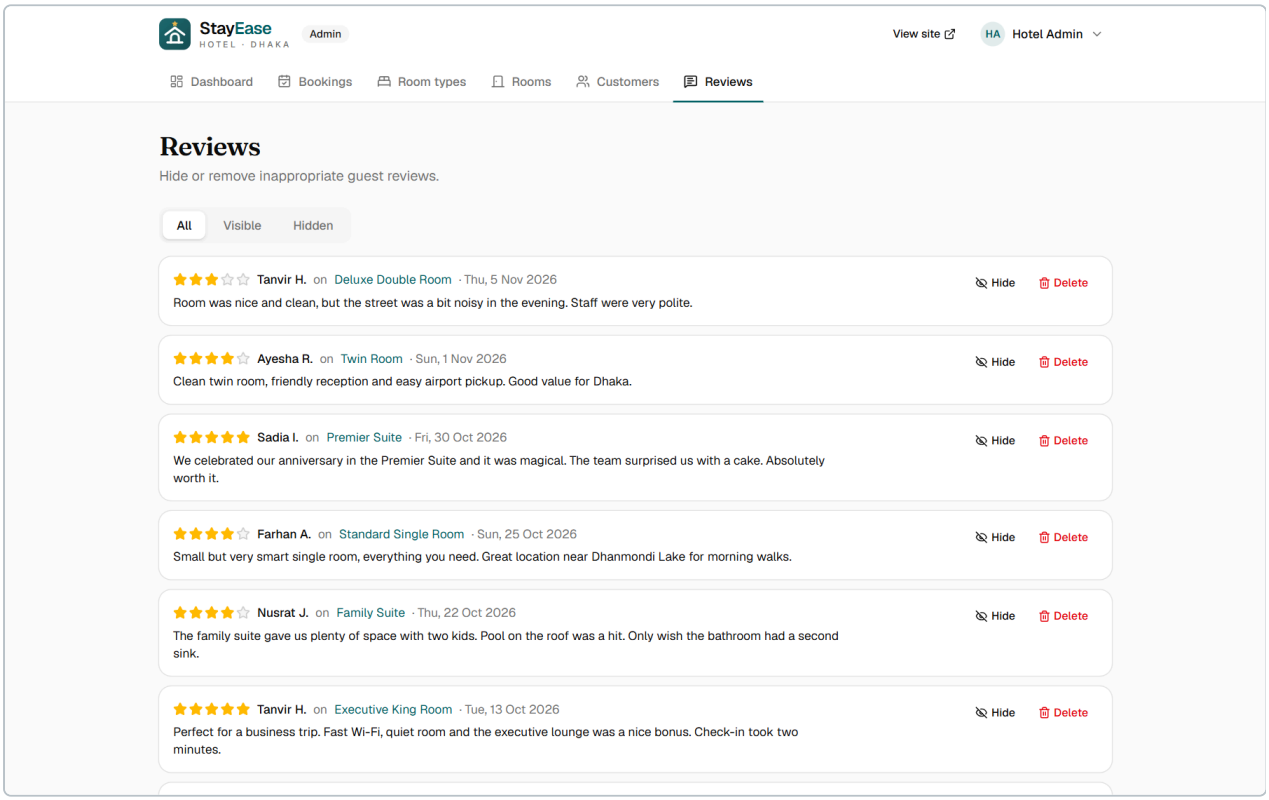


Figure 4.17: Review moderation

4.8.4 Mobile view

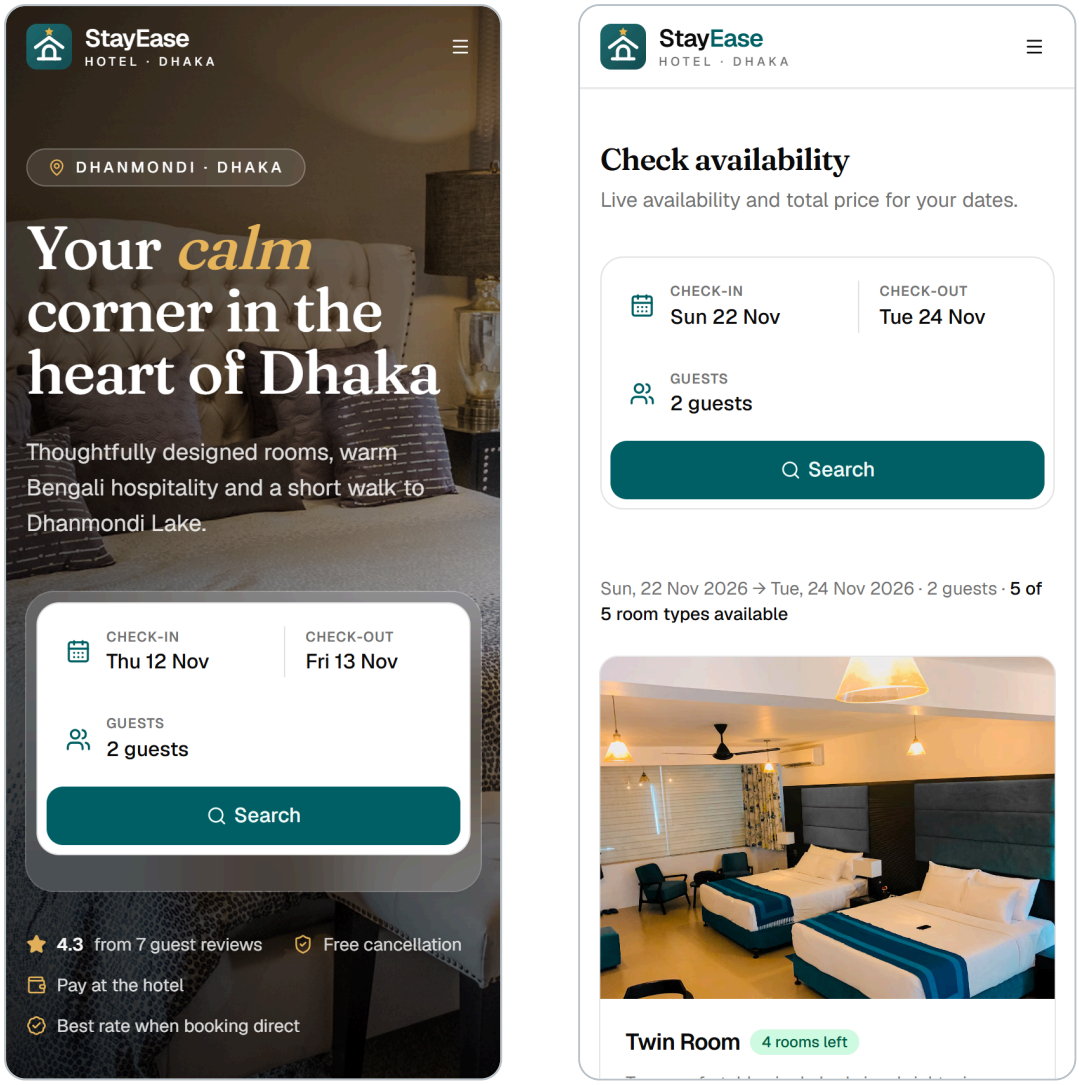


Figure 4.18: Responsive design on a mobile phone: home page and search results

4.9 Conclusion

StayEase meets the objectives set in Chapter 1. Guests can find and book a room online in minutes, the hotel manages all reservations from one dashboard, and double bookings are prevented by design and proven by automated tests. The system is live on the internet and can be extended in the future with online payment, e-mail notifications, seasonal pricing and multi-hotel support.